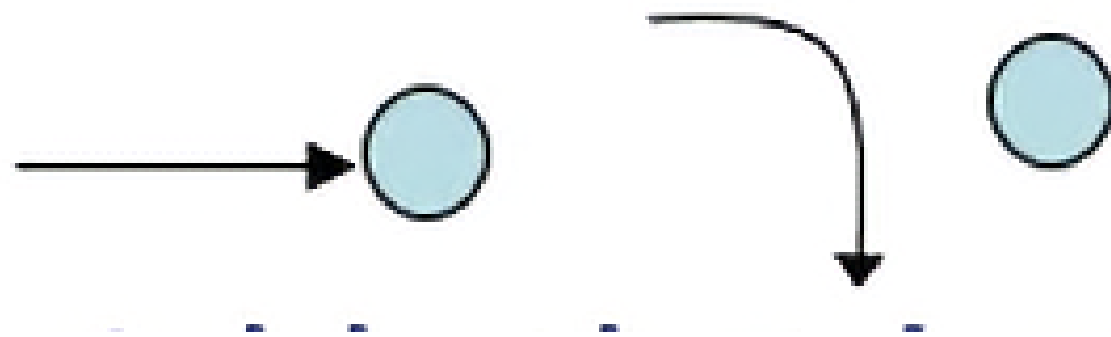


Garbage Collection

이주석

Garbage Collection

```
char *p;  
p = (char *)malloc(100);  
...  
free(p);
```

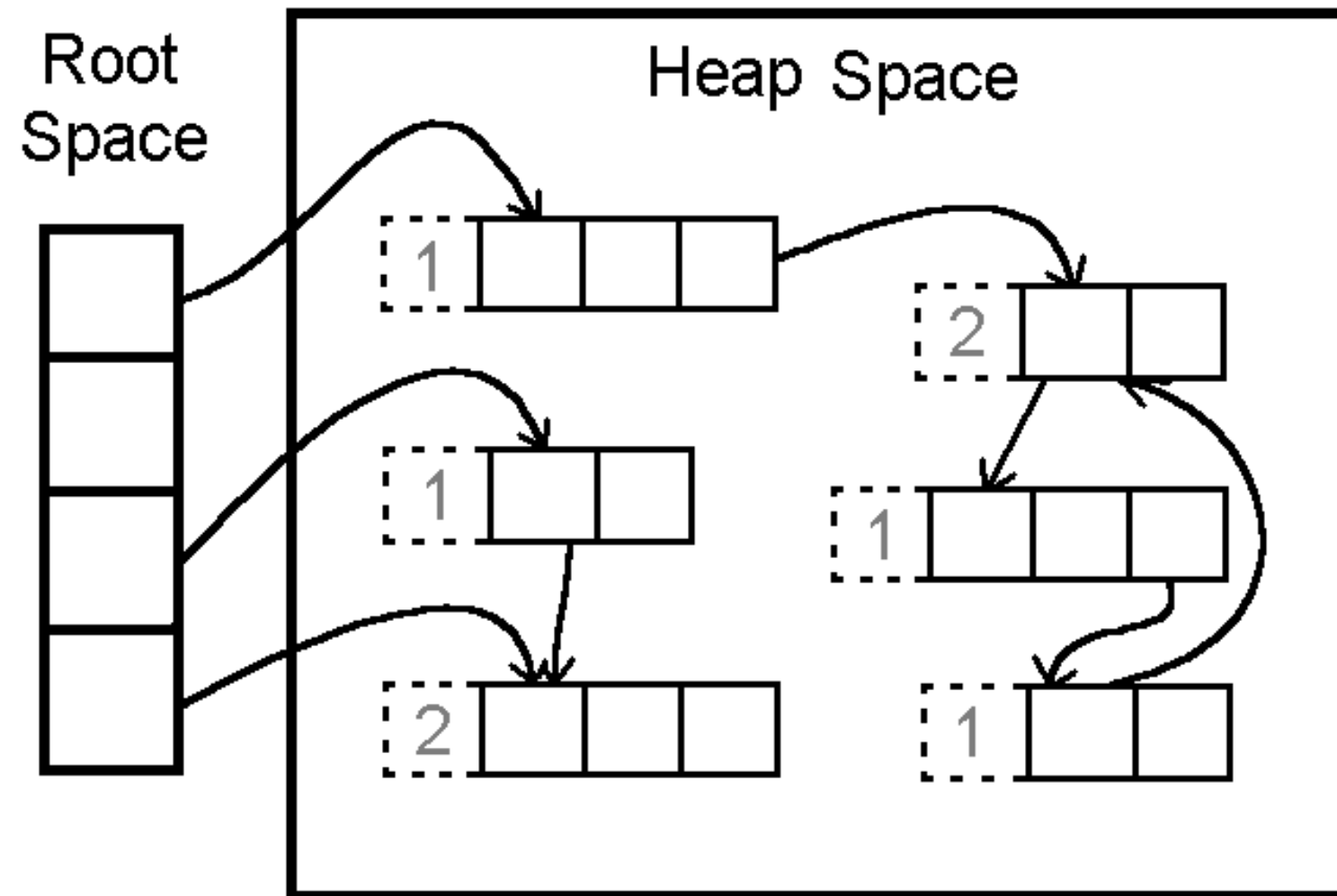


```
Object foo = new Object()  
foo = new Object();
```

- C에서는 직접 malloc, new 할당후 free 또는 delete
 - dangling pointer로 인한 메모리 누수.. 그 외 악명 높은 포인터의 어려움..
 - 다행히? 자바에선 new 로 생성 후 해제가 없다
 - 메모리 해제는 개발자가 코드로 하는 것이 아니라, 오로지 Garbage Collection 시스템이 수행한다
- GC: 필요없는(참조되지 않는) 객체들을 해제하는것

Reference Counter

python, swift



- 필요없는(참조되지 않는) 객체들을 어떻게 아나?
→ 참조하는 횟수를 센다.

1. 각 객체들이 나를 얼마나 참조하는지 reference count로 횟수를 관리한다.
2. reference count가 0이 되면 자동으로 해제한다.

장점:

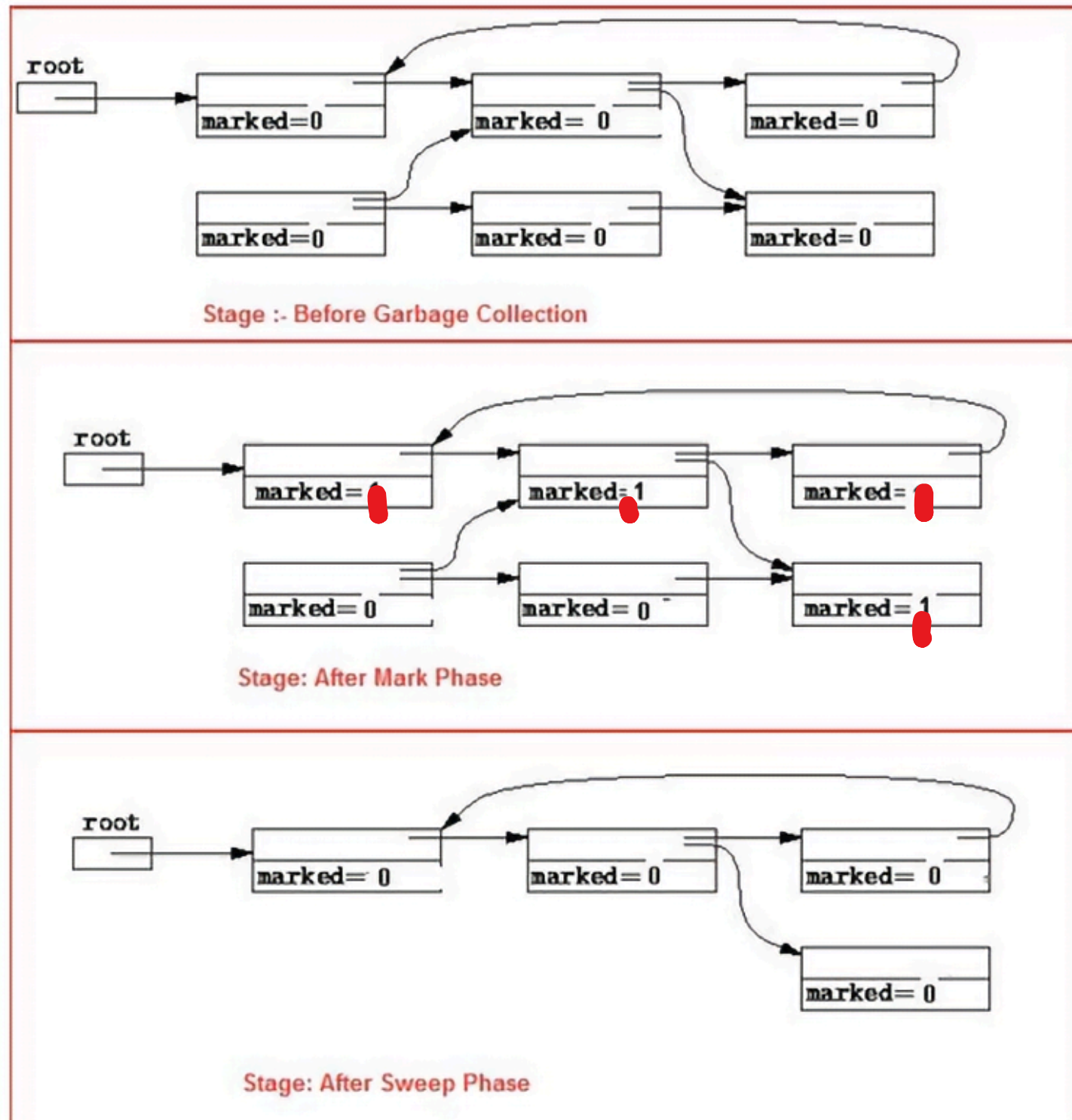
- 0 되면 해제되므로 점진적으로 일어남

단점:

- 추가적인 메모리
- 참조, 해제마다 체크
- **순환 참조 문제 해결해야함**

MARK and SWEEP

Java, C#, JavaScript.. 많은 언어



- 필요없는(참조되지 않는) 객체들을 어떻게 아나?

1. 메모리를 참조하는 root를 둔다.
2. GC가 필요해졌을때 root로 부터 도달 가능한지 체크한다.

- 그래프 탐색으로 찾을 수 있음

3. 도달 못했으면(unreachable)? 해제하면 됨

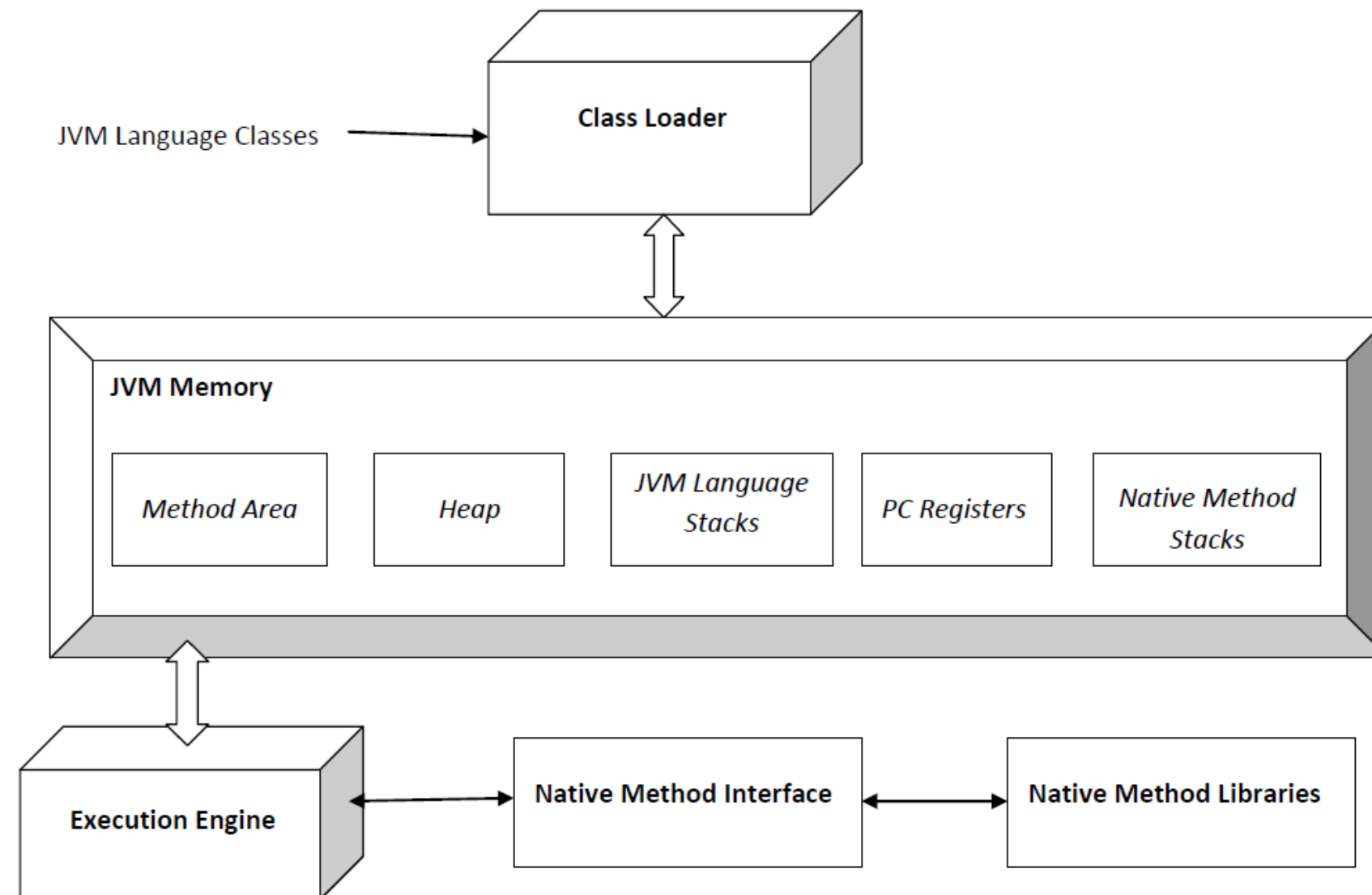
장점:

- 순환참조 없음

단점:

- 프로그램 일시정지(stop the world)

JVM



- JVM: 자바?로 개발한 프로그램을 컴파일하여 만들어지는 바이트코드 실행시키기 위한 가상머신.

크게 3가지로 구분됨

- 클래스 로더: 바이트코드(.class파일들) 읽어서 메모리에 적재
- 메모리 영역(Runtime Data Area): 상수 풀(constant pool), Heap 등이 있음
- 실행 엔진: 바이트코드를 실행하는 JIT컴파일러, heap의 메모리를 해제하는 **Garbage Collector**
- 하나하나 내용이 매우많기에... Garbage Collector라도 잘 알자!

Implementation details that are not part of the Java Virtual Machine's specification
<https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-2.html>

???

JVM

Implementation details that are not part of the Java Virtual Machine's specification

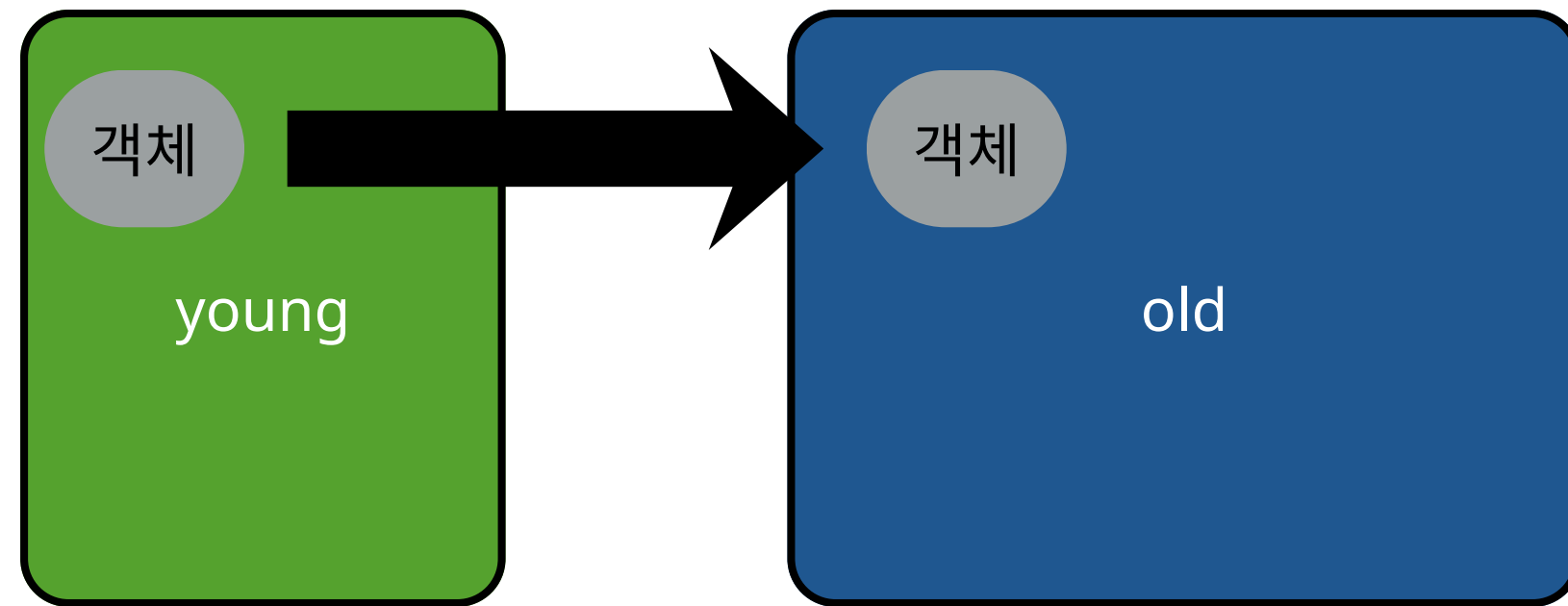
???

- JVM 스펙에는 "힙(Heap)을 어떻게 구성해라"라는 구체적인 정의가 없음
- 가비지 콜렉터도 마찬가지. 어떻게 구현할지는 벤더 마음(심지어JIT까지)
- 즉, 자바의 GC는 어떤 버전이냐, 어떤 벤더냐에 따라 달라짐
- 대신 보통 Oracle의 Hotspot JVM 이므로 인터넷에 있는 자료도 이 JVM기준이다.

→ 앞의 두가지는 범용적인 CS지식이니 알아야 할 것이고..

- 자바 GC들을 일일이 다 외우는 것은 비추천
- 하지만 뒤에 나올 Generational garbage collection 방식 정도는 알아두는것이 좋다.

Generational GC

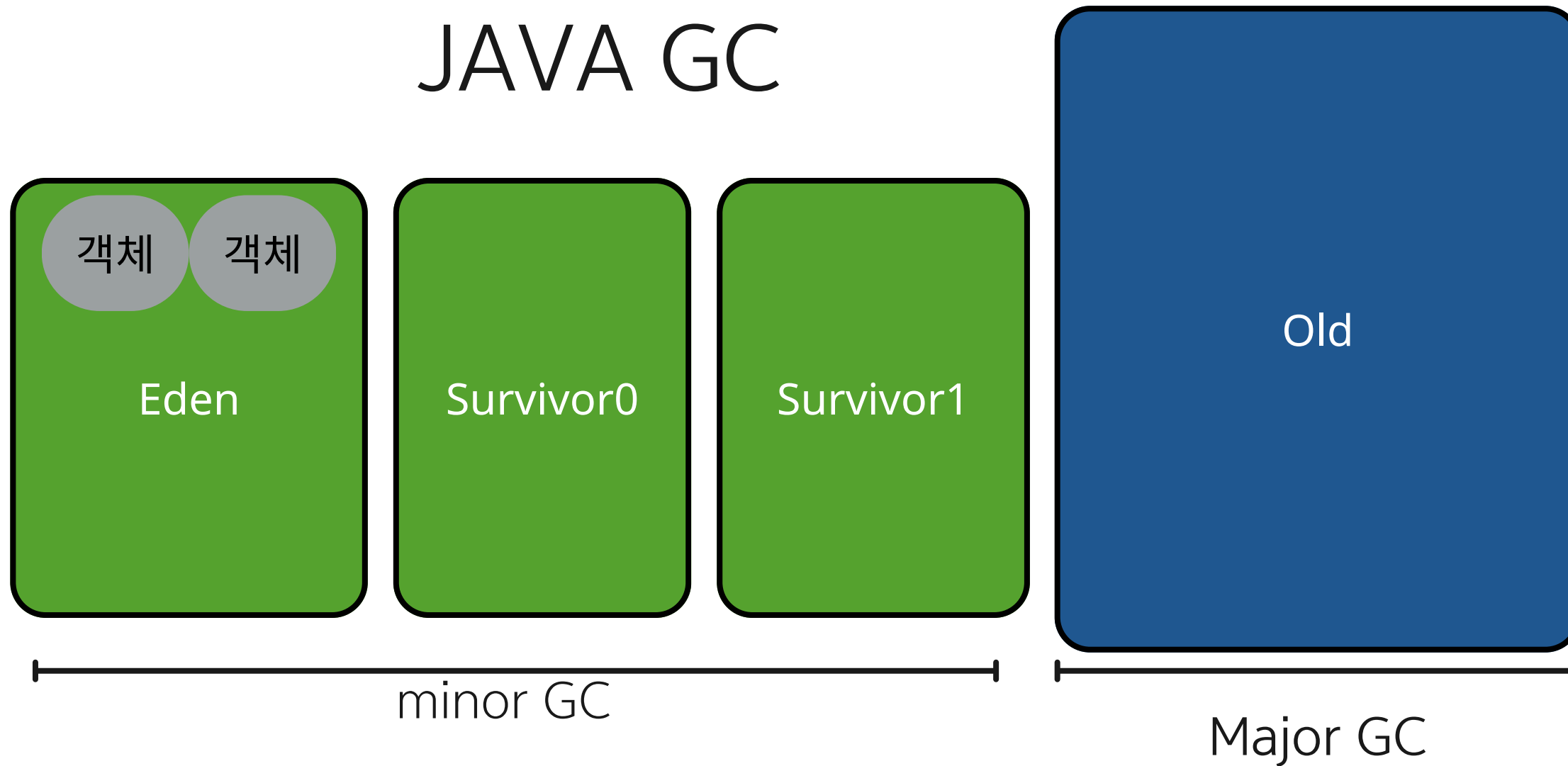


경험적으로 객체는 금방 garbage가 된다.

->Heap을 2가지 generation으로 영역구분

- Young generation
- Old generation
- GC를 실행 할때마다 age를 증가시키고 일정수준이되면 옮긴다(promotion)

JAVA GC



Young generation을 3가지로 나눈다

- Eden : 할당을 위한 공간
- Survivor 0,1: eden에서 살아남은 객체들
survivor에서도 살아남으면 다른 survivor로 이동

- Eden 영역이 가득차면 survivor 영역으로 옮긴다.
이때 하나의 survivor 영역으로만 옮긴다.
- survivor0 → survivor1 → survivor0...
- 이렇게 [Eden에 살아남은 객체] + [Survivor에 있던 객체]가 다른 Survivor로 이동한다.
- 이 과정에서 age가 일정 기준을 넘으면 old generation으로 옮긴다 (=promotion)
- Old promotion마저 꽉차면? Major GC가 발생한다. (긴 정지시간)

요약

GC: 참조되지 않는 객체들을 자동으로 해제

장점:

- 개발자 실수 방지: 해제된 메모리 접근, 같은 메모리 2번 해제
- 메모리 누수 없음

단점:

- 언제 할지 모름.
- GC 오버헤드(stop the world)

방식:

- Reference counter
- Mark and Sweep

Generational GC:

young generation과 old generation으로 나눈다

- minor GC를 반복하며 오래 살아남은 객체를 old generation으로 보내준다.
- old마져 가득차면 major GC가 일어난다

Q&A

질의 응답

